

API SECURITY RISK ANALYSIS REPORT

Cyber Security Task 3 – Future Interns (2026)

Submitted By: DHARANI V

Internship Program: Future Interns – Cyber Security Task 3

Project Title: API Security Risk Analysis using JSONPlaceholder API

Submission Date: May 2026

Tools Used : Postman,Browser Devop Tools,Google Chrome

Introduction:

Modern applications heavily rely on APIs (Application Programming Interfaces) for communication between mobile applications, websites, SaaS platforms, and backend services. APIs transfer sensitive information and perform business-critical operations. Therefore, securing APIs is essential to prevent unauthorized access, data exposure, and system abuse.

This project focuses on conducting a **read-only API Security Risk Analysis** using a public testing API. The objective was to inspect endpoints, analyze authentication and authorization mechanisms, review response data, and identify potential API security risks in an ethical manner.

Objective:

The objective of this task is to:

- Analyze public/demo APIs safely
- Identify common API security risks
- Review authentication and authorization mechanisms
- Inspect API responses and headers
- Assess security posture
- Suggest remediation recommendations
- Document findings professionally

Scope of Analysis:

The analysis was conducted under ethical guidelines and restricted to:

Allowed Activities

Read-only API testing Documentation-

based analysis Header inspection

Response analysis

Endpoint testing

Restricted Activities

- No exploitation attempts
- No authentication bypass
- No brute force testing
- No Denial-of-Service testing
- No private API testing

The analysis was performed only on a public demo API intended for testing and learning.

Tools Used:

The following tools were used during testing:

Tool	Purpose
Postman	API request testing and response analysis
Browser	Endpoint inspection
JSONPlaceholder API	Public API for testing
Browser Developer Tools	Header and request observation
GitHub	Project documentation and submission

API Selected for Analysis:

APIName:

JSONPlaceholder API

Base URL: <https://jsonplaceholder.typicode.com>

Purpose:
JSONPlaceholder is a free fake REST API designed for testing and prototyping. It provides sample resources such as:

-
- Users
 - Posts
 - Photos
 - Comments

- Albums

The API is intentionally public and suitable for educational testing.

Testing Methodology:

The following methodology was followed during analysis:

Step 1: API Endpoint Testing

Endpoints such as:

- /users
- /photos
- /users/3
- /users?id=2

were tested.

Step 2: Authentication Review

The Authorization section in Postman was checked to determine whether authentication tokens or credentials were required.

Step 3: Header Analysis

HTTP response headers were inspected to understand rate limiting, server information, and security configurations.

Step 4: Input Validation Testing

Invalid endpoint testing was performed using random input to observe API behavior and response handling.

Step 5: Documentation of Findings

All observations were documented and supported using screenshots collected during testing.

Findings & Risk Analysis:

Finding 1: Open API Endpoint Access:

Observation:

The /users endpoint was accessible without any login credentials, API token, or authorization.

Example Endpoint:

GET /users

Risk Type:
Open / Unauthenticated Endpoint

Severity
Medium

Business Impact:

Unauthenticated access may expose user-related information to unauthorized individuals. In real-world SaaS systems, this may result in privacy risks and information leakage.

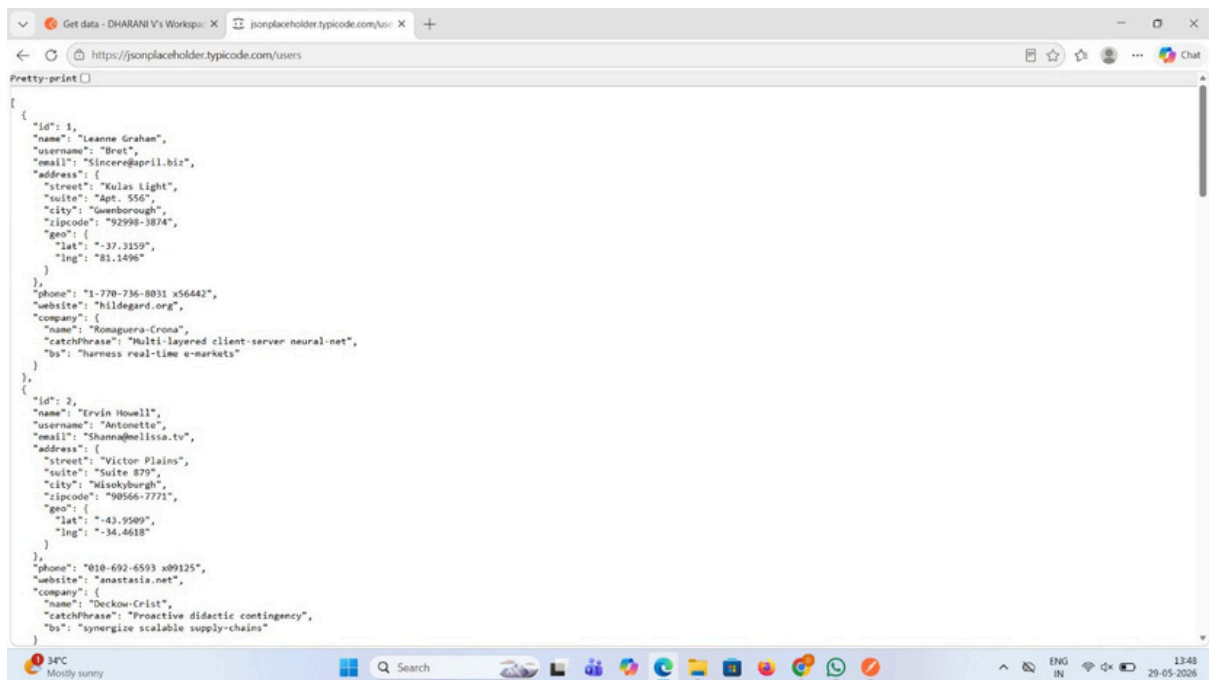
Recommendation:

Implement secure authentication methods such as:

- API Tokens
- OAuth 2.0
- JWT Authentication

Sensitive endpoints should require proper authentication.

Screenshots:



Finding 2: Excessive Data Exposure:

Observation:

The API response returned excessive user information including:

- Email
- Phone number
- Address details
- Geo-location
- Company information

Risk Type
Excessive Data Exposure
Severity
Medium

Business Impact:

In real production systems, exposing unnecessary user information can increase privacy risks and create opportunities for attackers to gather intelligence.

Recommendation

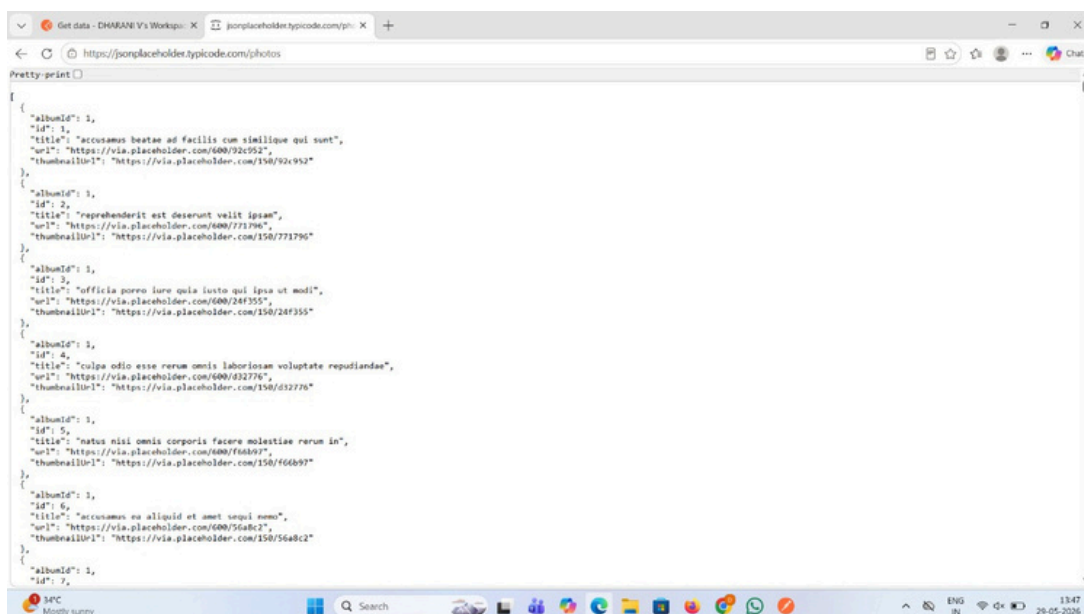
Return only required fields based on business necessity.

Example:

Instead of returning complete user profiles, expose only essential

Screenshots:

information.



```
{
  "albumId": 1,
  "id": 1,
  "title": "accusamus beatae ad facilis cum similique qui sunt",
  "url": "https://via.placeholder.com/600/92c952",
  "thumbnailUrl": "https://via.placeholder.com/150/92c952"
},
{
  "albumId": 1,
  "id": 2,
  "title": "reprehenderit est deserunt velit ipsam",
  "url": "https://via.placeholder.com/600/771790",
  "thumbnailUrl": "https://via.placeholder.com/150/771790"
},
{
  "albumId": 1,
  "id": 3,
  "title": "officia porro iure quia lusto qui ipsa ut modi",
  "url": "https://via.placeholder.com/600/24f355",
  "thumbnailUrl": "https://via.placeholder.com/150/24f355"
},
{
  "albumId": 1,
  "id": 4,
  "title": "culpa odio esse rerum omnis laboriosam voluptate repudiandae",
  "url": "https://via.placeholder.com/600/d32776",
  "thumbnailUrl": "https://via.placeholder.com/150/d32776"
},
{
  "albumId": 1,
  "id": 5,
  "title": "natus nisi omnis corporis facere molestiae rerum in",
  "url": "https://via.placeholder.com/600/f68997",
  "thumbnailUrl": "https://via.placeholder.com/150/f68997"
},
{
  "albumId": 1,
  "id": 6,
  "title": "accusamus eu aliquam et amet consequi nemo",
  "url": "https://via.placeholder.com/600/56a8c2",
  "thumbnailUrl": "https://via.placeholder.com/150/56a8c2"
},
{
  "albumId": 1,
  "id": 7,
  "title": "natus nisi omnis corporis facere molestiae rerum in",
  "url": "https://via.placeholder.com/600/24f355",
  "thumbnailUrl": "https://via.placeholder.com/150/24f355"
}
```

Finding 3: Authorization Weakness (Potential IDOR Risk):

Observation:

Specific user data was directly accessible using:

/users/3

The API returned detailed user information without verifying ownership or permissions.

Risk Type
Broken Object Level Authorization (Potential IDOR)

Severity
Medium

Business Impact:

In real-world systems, attackers may manipulate user IDs to access another user's private information.

Example:

/users/1

changing to

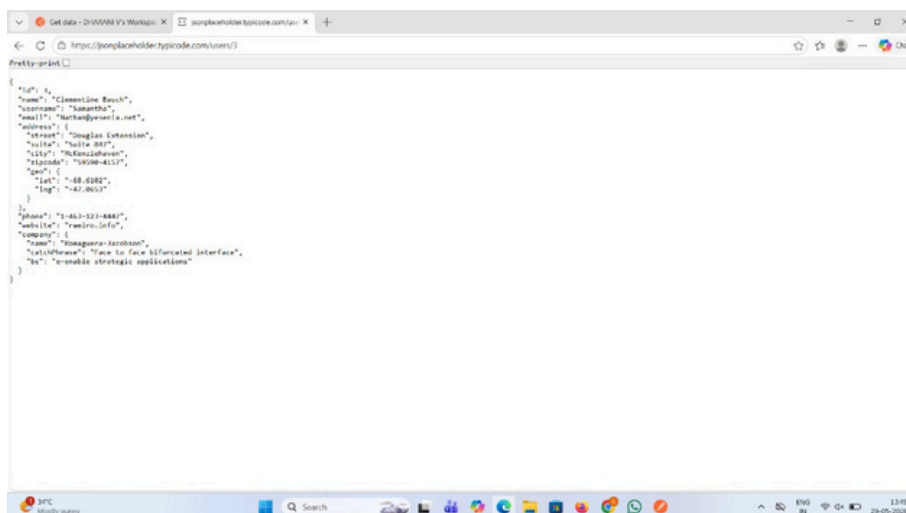
/users/2

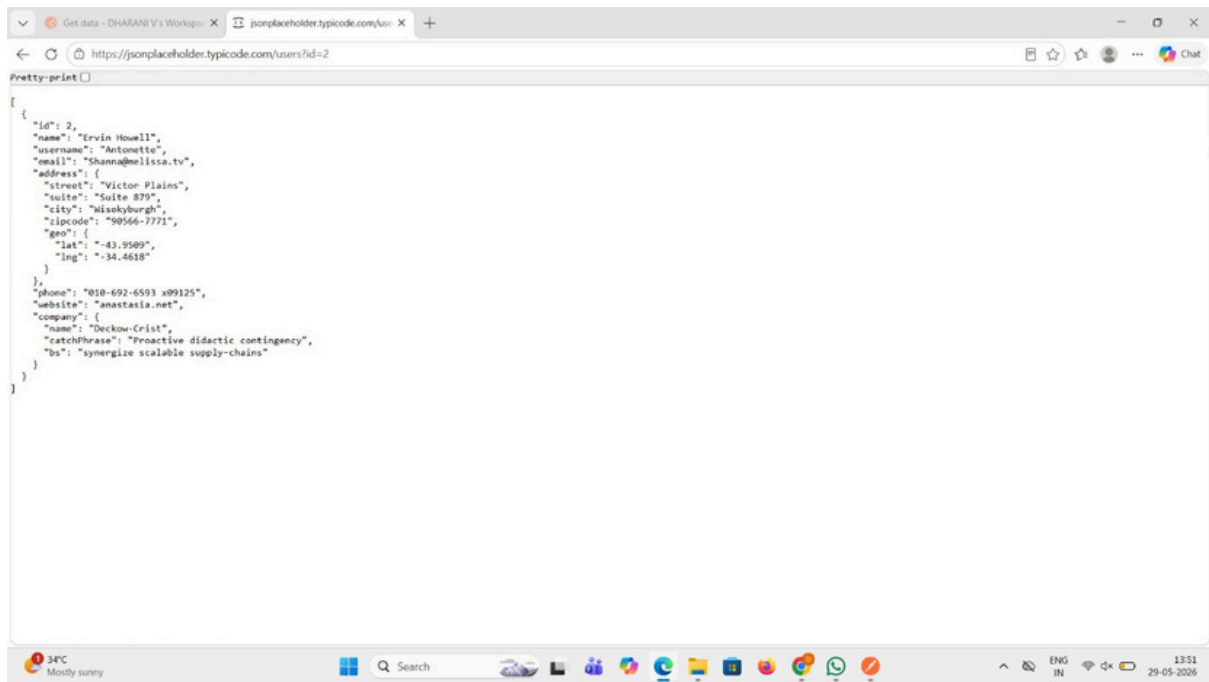
could expose another user's data.

Recommendation:

Implement authorization checks on the server-side to validate user ownership before exposing data.

Screenshots:





Finding 4: Rate Limiting Present (Positive Security Practice):

Observation

The response headers included:

x-ratelimit-limit = 1000

This indicates request limiting is configured.

Risk Type:
Security Strength

Severity

Positive Observation

Business Impact:

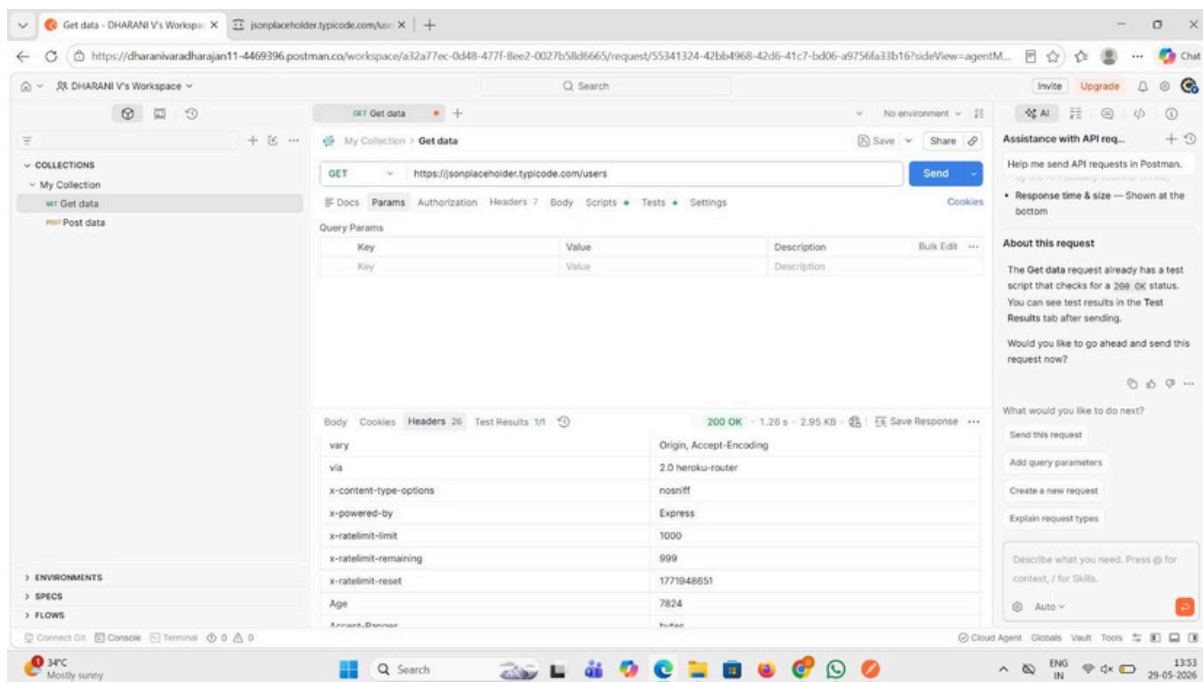
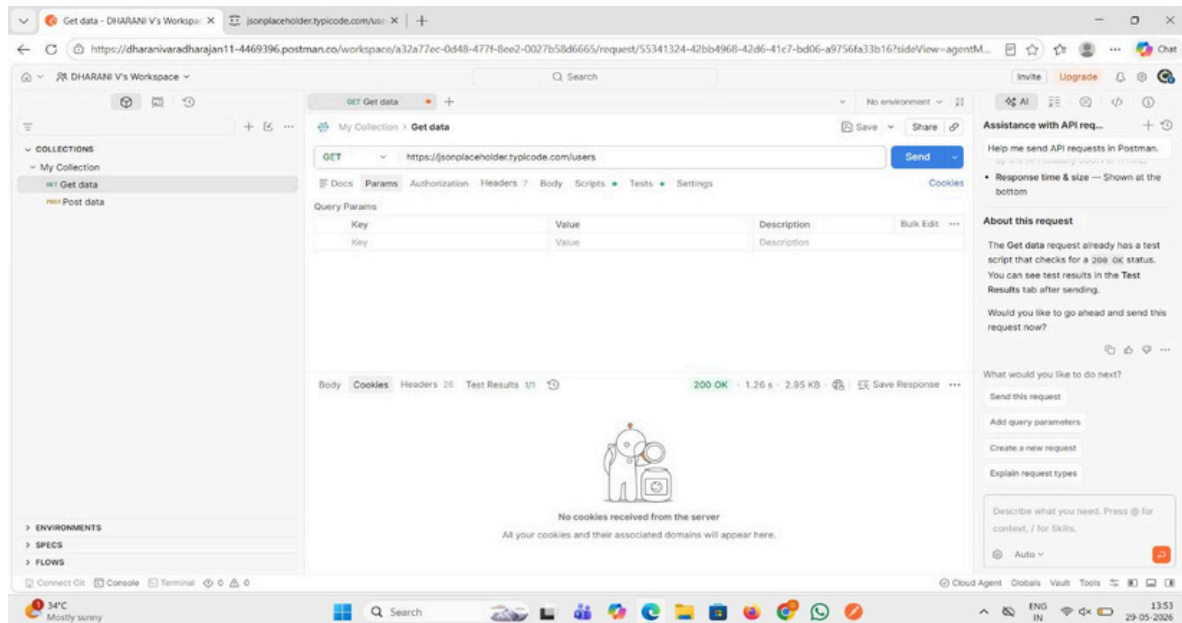
Rate limiting helps prevent:

- API abuse
- Excessive requests
- Brute-force attacks
- Server overload

Recommendation:

Continue enforcing request throttling and monitoring suspicious traffic.

Screenshots:



Finding 5: Input Validation Observation:

Observation:

An invalid endpoint was tested:

/dfgh

The API returned an empty response instead of crashing or exposing internal server details.

Risk Type:

Input Validation Observation

Severity

Low

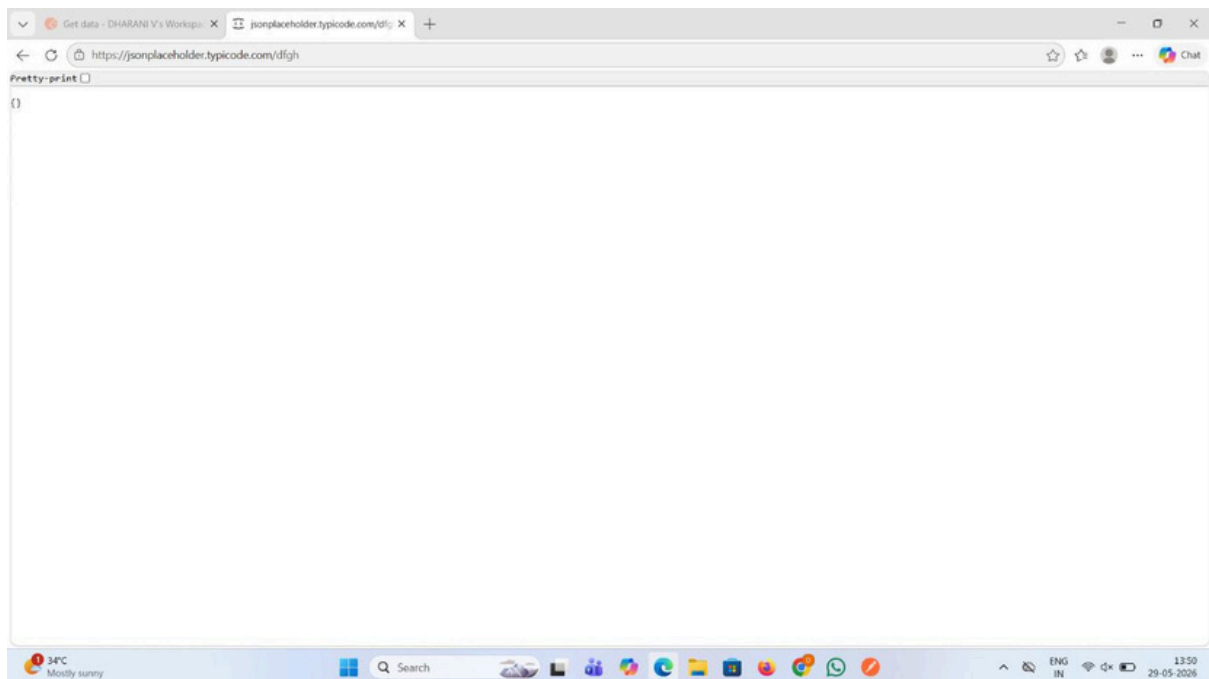
Business Impact:

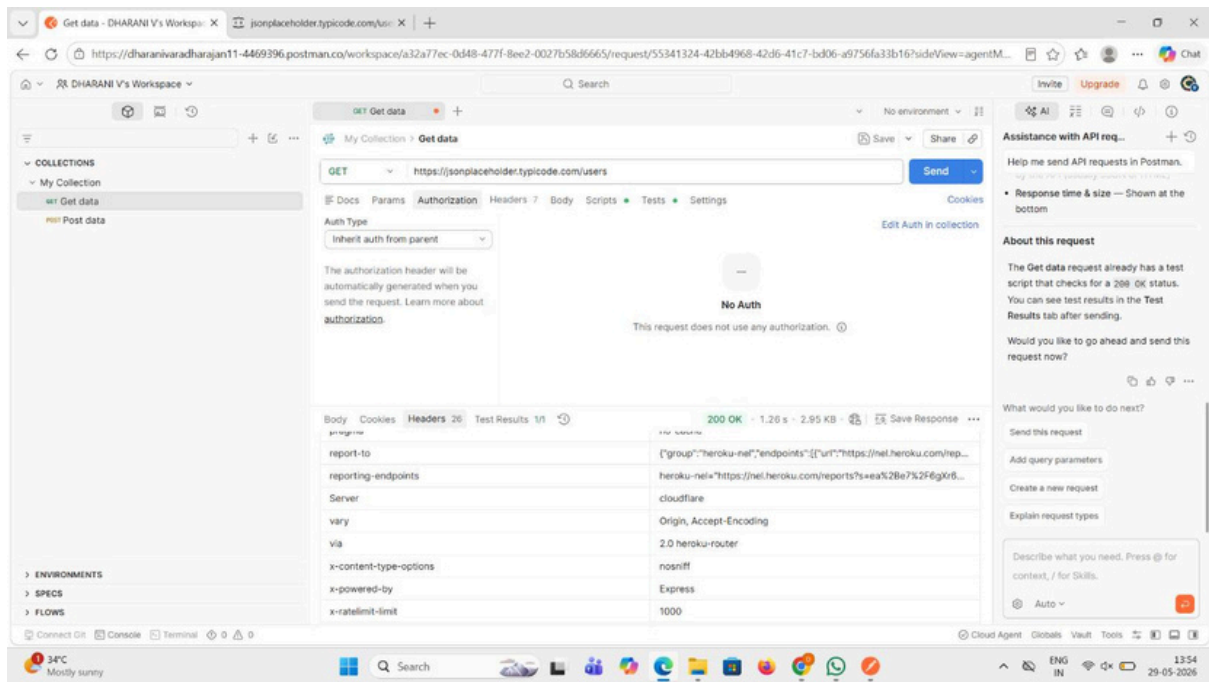
Proper handling of invalid inputs reduces unexpected behavior and improves system stability.

Recommendation:

Continue implementing proper error handling with standardized HTTP status responses.

Screenshots:





Risk Summary Table:

Risk	Severity	Business Impact	Recommendation
Open Endpoint Access	Medium	Unauthorized access	Add authentication
Excessive Data Exposure	Medium	Information leakage	Restrict unnecessary fields
Authorization Weakness	Medium	Unauthorized user access	Validate permissions
Rate Limiting Present	Positive	Protection against abuse	Continue enforcement
Input Validation Observation	Low	Better stability	Maintain validation

Conclusion:

This project successfully conducted a read-only API Security Risk Analysis using the JSONPlaceholder public API.

The analysis identified several important API security considerations including:

- Open endpoint accessibility
- Excessive data exposure
- Authorization concerns

- Rate limiting implementation
- Input validation behavior

Although JSONPlaceholder is intentionally public for educational purposes, the observations reflect real-world API risks that organizations must consider in production environments.

This task helped develop practical skills in:

- API inspection
- Security analysis
- Authentication assessment
- Risk identification
- Security documentation
- SaaS security fundamentals